

28. LLM_int8()

01 July 2026 11:31

LLM's require significant GPU memory for inference

Int8 matrix multiplication for feed forward and attention projection layers in transformers

↓
cuts the memory needed for inference by half while retaining full precision performance

LLM_int8() → A two part quantization procedure

- (i) use vector-wise quantization with separate normalization constants for each inner product in the matrix multiplication, to quantize most of the features
- (ii) for the emergent outliers, we use a mixed precision decomposition scheme, which isolates the outlier feature dimensions into a 16 bit matrix mult while still more than 99% of values are multiplied in 8-bit

abstract claims that higher precision model representations are converted into int8 representations for most of the matrix multiplication workload

for large transformers at & beyond 67B params

the feed forward, attention projection layers and mat-mults are responsible for 95% of consumed parameters and 65-85% of all computation

earlier attempts with quantization → reduce memory use but also degrade performance
limited to 30M param models

A Multi-billion-scale int8 quantization procedure for transformers that doesn't incur any performance degradation

Two challenges

- (i) The need for higher quantization precision at scales beyond 1B parameters
- (ii) Need to explicitly represent the sparse but systematic large

^{16 parameters}
(ii) Need to explicitly represent the sparse but systematic large magnitude outlier features that ruin quantization precision

For vector-wise quantization, mat mult can be seen as a sequence of independent inner products of row and column vectors

Use a separate quantization normalization constant for each inner product to improve quantization precision

Can recover the output by denormalizing before the next operation

→ Emergence of extreme outliers in the feature dimensions of hidden states during inference

for models as size increases these outliers appear in 25% of all transformer layers where large features with magnitudes upto 200x than in other dimensions

After at 67B scale it becomes severe reaching all transformer layers and 75% of all sequence dimensions

even if the count of these outliers is very low, changing them to zero, it hurts softmax attention probability and degrades validation perplexity

→ Mixed precision decomposition → 16 bit matrix multiplication for the outlier feature dimension
→ 8 bit matrix multiplication for the other 99% of dimensions

LLM.int8() → Vector wise quantization + Mixed precision decomposition

At which scale and why do quantization techniques fail?

And how is it related to quantization precision?

High-precision Asymmetric Quantization (Zero point Quantization)

Symmetric Quantization (absolute maximum quantization)

↓
most commonly used

↓
rarely used due to practical constraints

* summary

LLMs require large inference memory
Feed forward and attention projection layers dominate parameter storage
Matrix Multiplications dominate computation

Objective → Reduce memory usage, Use lower-precision matrix multiplication, Preserve model quality

Priorwork - transformer quantization Methods existed before, primarily studied models below 350M parameters

Performance degradation remains a concern, Scaling to billion params was unresolvd

Challenges 1 High quantization precision beyond 1B params
2 Extreme outlier features causing quantization problems

Vector wise quantization

Matrix multiplication is viewed as many independent inner products
Separate normalization constants are used
Outputs are recovered through de-normalization

Outlier Observations

- Can be more than 2x larger than typical feature dimensions
- Around 67B something changes
- The phenomenon becomes wide spread as model grows
- The outliers are systemic than Random

↓ phase shift (all transformer layers and 75% of sequence dimensions affected)

Remember the large model can be loaded, converted and used immediately without additional production degradation

Removing outlier dimensions

Causes (i) Large softmax probability changes → Removing random dimensions
(ii) Large perplexity degradation
↓
much smaller degradation

The outlier dimensions must be carrying information that is disproportionately

The outlier dimensions must be carrying information that is disproportionately important to model behavior

The emergent outliers are the key reason existing quantization methods fail at larger scales

8-bit Data types and Quantization

Abs max quantization scales inputs into the 8-bit range $[-127, 127]$ by multiplying with $s_{x_{f16}}$ which is 127 divided by the absolute maximum of the entire tensor

This is equivalent to dividing by the infinity norm and multiplying by 127

for an FP16 input matrix $X_{f16} \in \mathbb{R}^{s \times h}$ Int8 abs max quantization is

$$X_{i8} = \left\lfloor \frac{127 X_{f16}}{\max_{i,j} (|X_{f16,ij}|)} \right\rfloor \xrightarrow{\text{rounding off to nearest integer}} \left\lfloor \frac{127}{\|X_{f16}\|_{\infty}} X_{f16} \right\rfloor = \left\lfloor s_{x_{f16}} X_{f16} \right\rfloor$$

Zero point Quantization

↳ Shifts the input distribution into the full range $[-127, 127]$ by scaling with the normalized dynamic range ndr_x and then shifting by the zero point zp_x

This affine transformation, any input tensors will use bits of the data type thus reducing the quantization error for asymmetric distributions

$$ndr_{x_{f16}} = \frac{2 \cdot 127}{\max_{i,j} (X_{f16}^{ij}) - \min_{i,j} (X_{f16}^{ij})}$$

$$zp_{x_{i16}} = \left\lfloor X_{f16} \min_{i,j} (X_{f16}^{ij}) \right\rfloor$$

$$X_{i8} = \left\lfloor ndr_{x_{f16}} X_{f16} \right\rfloor$$

To use this quantization in an operation we feed both X_{i8} and the zero point $zp_{x_{i16}}$

To use this quantization in an operation we feed both X_{18} and the zero point $ZP_{x,16}$ into a special instruction which adds $ZP_{x,16}$ to each element of X_{18} before performing a 16 bit integer operation

eg to multiply two zeropoint quantized numbers A_{18} and B_{18} along with their zero points $ZP_{a,16}$ and $ZP_{b,16}$

$$C_{132} = \text{multiply}_{16} (A_{ZP_{a,16}}, B_{ZP_{b,16}}) = (A_{18} + ZP_{a,16})(B_{18} + ZP_{b,16})$$

When rounding is required if the instruction multiply_{16} is not available on GPUs/TPUs

$$C_{132} = A_{18} B_{18} + A_{18} ZP_{b,16} + B_{18} ZP_{a,16} + ZP_{a,16} ZP_{b,16}$$

$\underbrace{\hspace{10em}}$
 computed with 16 bit precision with int 16/32 precision

In Both cases outputs are accumulated as 32 bit integer C_{32}

To dequantize C_{32} we divide by scaling constants $rd_{a,16}$, $rd_{b,16}$

Int8 Matrix Multiplication with 16 bit Float inputs and Outputs

hidden states	$X_{f,16} \in \mathbb{R}^{s \times h}$	s sequence dimension
Weights	$W_{f,16} \in \mathbb{R}^{h \times o}$	h feature dimension
		o output dimension

We perform 8 bit matmul with 16 bit input & outputs as

$$X_{f,16} W_{f,16} = C_{f,16} \approx \frac{1}{C_{x,f,16} C_{w,f,16}} C_{132} = S_{f,16} C_{132}$$

$$\approx S_{f,16} A_{18} B_{18} = S_{f,16} Q(A_{f,16}) Q(B_{f,16})$$

Q is either abs max or zeropoint quantization

$C_{x,f,16}$ and $C_{w,f,16}$ are respective tensor wise scaling constants S_x and S_w for abs max

and rd_x and rd_w for zeropoint

absolute scaling constant is $S_{x_{f16}} = \frac{127}{\max_{ij} |X'_{f16}|}$ quantization is $X_8 = \lfloor S_{x_{f16}} X_{f16} \rfloor$

Representation view $\rightarrow$ The largest magnitude value in the tensor becomes approximately ± 127
Everything else is scaled proportionally

why 2×127 ?
 $nd_x = \frac{2 \times 127}{\max(X) - \min(X)}$ target int8 range $[-127, 127]$ width is $127 - (-127) = 254 = 2 \times 127$
 $nd_x = \frac{\text{target width}}{\text{original width}}$

What is zero point doing?

$$zp_{x_{f16}} = \lfloor nd_{x_{f16}} \min(X_{f16}) \rfloor$$

Interpretation

After scaling, the minimum value of the original tensor lands somewhere on the integer axis
The zero-point term records that offset

Information flow
 step 1 (scale) $X \rightarrow nd_x X$
 step 2 (shift) $nd_x X \rightarrow nd_x X - zp$

Up until this section

The quantized tensor uses int8 values, while the corresponding zero-point parameter is represented as int16

$X_{f16} W_{f16} = C_{f16}$ float tensors $\rightarrow$ quantized $\rightarrow$ integer matrix multiply $\rightarrow$ dequantization using scaling constants $\rightarrow$ Approximate FP16 result

Information flow

$X_{f16} \rightarrow Q(X_{f16})$
 $W_{f16} \rightarrow Q(W_{f16})$
 $Q(X) Q(W) \rightarrow C_{i32}$
 $C_{i32} \rightarrow \text{scaled back} \rightarrow \text{FP16 approximation}$

Int8 Matrix Multiplication at Scale

single scaling constant if there is an outlier (it can reduce quantization precision of all other values)

↓

Hence multiple scaling constants per tensor to isolate outlier effects

Vector wise Quantization

→ view matrix multiplication as a sequence of independent inner products

→ Assigning a different scaling constant c_{xf16} to each row of X_{f16} and c_w to each column of W_{f16}

→ To dequantize, denormalize each inner product result by $1/(c_{xf16} c_{wf16})$

For the whole matrix this is equivalent to denormalization by outerproduct

$$C_{xf16} \otimes C_{wf16} \quad C_x \in \mathbb{R}^s \quad C_w \in \mathbb{R}^D$$

$$C_{f16} \approx \frac{1}{C_{xf16} \otimes C_{wf16}} C_{132} = S C_{132} = S A_{18} B_{18} = S Q(A_{f16}) Q(B_{f16})$$

Mixed Precision Decomposition

vector quantization → ineffective for outlier features

Systemic & Sparse

0.1% of all feature dimensions

Outliers occur systematically for almost all sequence dimensions 's' but are limited to specific feature/hidden dimensions 'h'

Separate Outlier feature dimensions into the set $O = \{i | i \in \mathbb{Z}, 0 \leq i \leq h\}$

which contains all dimensions of h which atleast have one outlier with a magnitude larger than the threshold α ($\alpha = 60$ suggested)

↓
to reduce performance degradation to zero close

Using Einstein notation where all indices are superscripts

Contribution from outlier dimensions kept in F16

FF. and A. ...

Using Einstein notation where all indices are superscripts close

The split is by feature dimensions and not individual values

MPD is

$$C_{f16} \approx \sum_{h \in D} X_{f16}^h W_{f16}^h + S_{f16} \sum_{h \in D} X_{i8}^h W_{i8}^h \rightarrow \text{computed in int8}$$

Contribution from outlier dimensions kept in F16

denormalization term for int8 inputs and weight matrices X_{i8} W_{i8}

Avoids high precision multiplication of outliers

Metrics in experimental setup

- (i) language modeling perplexity to compare different quantization baselines
 - ↳ very sensitive to quantization degradation
- (ii) Zero shot accuracy degradation for different end tasks

Summary & notes

Single scaling constant

If a tensor contains one large outlier $S = \frac{127}{\max(|X|)}$ must be chosen using that outlier

Consequences The outlier is represented well
many ordinary values get squeezed into fewer quantization levels

So quantization precision is lost for most values

Treat mat mults as sequence of independent inner products -

for $X_{f16} \in \mathbb{R}^{S \times h}$ and $W_{f16} \in \mathbb{R}^{h \times v}$

each output element $C_{ij} = \sum_k X_{ik} W_{kj}$

row i of X
inner product with
column j of W

Assigning diff scaling constant to each row of X and each column of W

$C_x \in \mathbb{R}^S$ contains one scaling constant per row

$C_w \in \mathbb{R}^v$ contains one scaling constant per column

$$S = \frac{1}{C_x \otimes C_w}$$

↳ outer product

eg if $C_x = \begin{bmatrix} a \\ b \end{bmatrix}$ $C_w = \begin{bmatrix} p & q \end{bmatrix}$

$$C_x \otimes C_w = \begin{bmatrix} ap & aq \\ bp & bq \end{bmatrix}$$

So every output receives its own dequantization factor

Outliers sparsity is over feature dimensions (o.i.)

$$O = \{i \mid \exists z \in \mathbb{Z}, 0 \leq i \leq h, \}$$

all dimensions i for which atleast one outlier has a magnitude larger than α

O contains feature dimension indices

eg $h=4096$

only dimensions 17, 212, 9720 exhibit outliers

$$\text{Then } O = \{17, 212, 9720\}$$

4 Emergent large magnitude features in Transformers at Scale

outlier features strongly affect attention and overall predictive performance

eg for a 13B model 150 outliers per 2048 token sequence

they are systemic and at most represent $\rightarrow$ unique feature dimensions h



Occurs in most layers or not at all

in smaller models the outliers are probabilistic (occur sometimes in some layers of each sequence)

$X \in \mathbb{R}^{s \times h}$
 $\rightarrow$ sequence dimension
 $\rightarrow$ hidden dimension (feature dim)

$X = \begin{bmatrix} \text{token 1} \\ \vdots \\ \text{token n} \end{bmatrix}$ each row is one token representation
 $X_i \in \mathbb{R}^h$

s = Sequence dimension (no of token positions)

eg 2048 token context $s=2048$

h = feature dimension

The coordinate index inside a hidden vector

eg GPT hidden state $h=4096$

Then every token is represented by $[x_1, x_2, \dots, x_{4096}]$

each co-ordinate is a feature dimension

What's an outlier?

They don't say particular token is an outlier!

They say particular co-ordinates become outliers

eg suppose dimension 172 is huge everywhere

$$x_{172} = 40, 50, 70, 70$$

while most dimensions are -1 to 1

Then dimension 172 is an outlier feature dimension

How are they tracking outliers?

Choose feature dimension $\rightarrow$ then examine that SAME co-ordinate
across all layers
all sequence positions

eg suppose $h=4096$

Choose h_{172}

Now inspect Layer 1 token₁, token₂, token₃

Layer 2 token₁, token₂, token₃

They track dimension 172 everywhere

Then they ask if this coordinate exceed $\alpha=6$ enough

$\rightarrow$ Outlier feature must appear in atleast 25% of layers

eg if $L=48$ then $25\% = 12$

Dimension 172 must show an outlier in atleast 12 layers

6 / sequence dimensions ?

$S=2048$ tokens

then $0.06 \times 2048 = 123$ token positions

The same feature dimension must be an outlier, at least 128 token positions

They filter away rare accidents

They only keep feature dimensions that repeatedly become large } hence systems across many layers across many tokens

They analyse G, K, V, O projection, FFN expansion layers
'ignore' attention function and FFN contraction layer

A sharp jump between 6B and 67B 65% $\rightarrow$ 100%

Quantization bins become empty and values get quantized to zero

↓

Suppose normal values are $[-1, 1]$

now an outlier appears 60

Abnormal Scaling becomes $\frac{127}{60}$

Instead of $\frac{127}{1}$

Now many ordinary values collapse into very few quantization bins

eg 0.01
might become 0
after rounding 0.02
might become 0
⋮
so on

Information disappears

Many different values become identical

} meant by
extinguishing information

Why does quantization fail beyond 67B?

The paper's argument

- I Large outlier dimensions emerge
- II Magnitude becomes very large

- i Large outlier dimensions emerge
- ii Magnitude becomes very large
- iii Quantization range expands
- iv Most bins become unused
- v Ordinary values collapse
- vi Prediction quality degrades

Why row-wise and vector-wise still fail?

Row-wise scaling operates along $\rightarrow$ sequence dimension
 Vector-wise scaling also scales $\rightarrow$ rows and columns

But the outliers occur along feature dimensions

Those are different axes

The scaling strategy is operating on one direction

The pathology lives in another direction

So, even improved scaling cannot fully isolate those outlier dimensions

Hence, the requirement for mixed precision decomposition

Why Zero point helps?

Outliers are often mostly positive or mostly negative

that means distribution is asymmetric and not centered around zero

Absmax quantization assumes symmetry unlike zero-point

Hence zero-point can represent outlier distributions better

Section 5 Related work

8 bit datatypes

INT8 supported efficiently by GPUs
 hardware friendly
 limited dynamic range

FP8 potentially better numerical representation
 not widely supported at the time

Competing Quantization Approaches

Group wise quantization finer granularity and higher precision

trade off requires custom CUDA kernels

LLM int8() slightly coarser
simpler deployment
no custom kernels

Limitations

- 1 Only int8 studied no FP8
- 2 only upto 175B parameters Larger models may exhibit new phenomena
- 3 Attention Scores themselves were not quantized Only projection matrices
- 4 Inference only, not training, not fine-tuning

Key finding

Model complexity/perplexity correlates with outlier emergence more smoothly than parameter count

The 67B phase shift appears abrupt in parameter space but smoother in perplexity space